

LAPORAN PRAKTIKUM
STRUKTUR DATA
“DOUBLY LINKED LIST”
PADA BAHASA PEMROGRAMAN JAVA
disusun Oleh :
RAZIF AL FARISI
2511532028

Dosen Pengampu : Dr. Wahyudi S.T, M.T
Asisten Laboratorium : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul **“Doubly Linked List pada Bahasa Pemrograman Java”**. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum pada mata kuliah Struktur Data, khususnya mengenai implementasi Doubly Linked List pada bahasa pemrograman Java.

Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai konsep linked list, khususnya Doubly Linked List, cara kerja node, pointer, serta operasi-operasi dasar yang dapat dilakukan seperti insertion, deletion, traversal, dan searching.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bimbingan, bantuan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data yang telah memberikan ilmu dan arahan selama proses pembelajaran.

Akhir kata, penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa mendatang.

Padang, 15 Mei 2026

Penulis

Daftar Isi

KATA PENGANTAR	i
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan	3
BAB II	4
PEMBAHASAN	4
2.1 Landasan Teori	4
2.2 Implementasi Program	4
2.2.1 Class NodeDLL_2511532028.....	4
2.2.2 Class InsertDLL_2511532028.....	5
2.2.3 Class HapusDLL_2511532028	11
2.2.4 Class PenelusuranDLL_2511532028	16
2.3 Tugas Praktikum	19
2.3.1 Class Lagu_2511532028	19
2.3.2 Class Musik_2511532028	20
BAB III.....	32
PENUTUP.....	32
3.1 Kesimpulan.....	32
3.2 Saran	32
Referensi.....	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu komponen penting dalam dunia pemrograman yang digunakan untuk mengatur, menyimpan, serta mengelola data secara efisien. Pemilihan struktur data yang tepat sangat berpengaruh terhadap performa suatu program, terutama dalam proses manipulasi data seperti penambahan, penghapusan, pencarian, maupun penelusuran data.

Salah satu struktur data linear yang sering digunakan adalah linked list. Linked list memiliki sifat dinamis, sehingga ukuran data dapat bertambah maupun berkurang sesuai kebutuhan tanpa harus menentukan kapasitas di awal seperti pada array.

Doubly Linked List merupakan pengembangan dari Single Linked List, di mana setiap node tidak hanya memiliki pointer yang menunjuk ke node berikutnya, tetapi juga memiliki pointer yang menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, proses traversal dapat dilakukan ke dua arah, yaitu maju maupun mundur, sehingga memberikan fleksibilitas yang lebih tinggi dalam pengolahan data.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Doubly Linked List serta mampu mengimplementasikan berbagai operasi dasar seperti insertion, deletion, searching, dan traversal menggunakan bahasa pemrograman Java.

1.2 Tujuan

Tujuan dari penyusunan laporan praktikum ini adalah sebagai berikut:

1. Memahami konsep Doubly Linked List dalam struktur data.
2. Memahami struktur node pada Doubly Linked List.
3. Mengimplementasikan operasi insertion pada Doubly Linked List.
4. Mengimplementasikan operasi searching pada Doubly Linked List.
5. Mengimplementasikan operasi deletion pada Doubly Linked List.
6. Memahami proses traversal maju maupun traversal mundur pada Doubly Linked List.
7. Mengimplementasikan Doubly Linked List menggunakan bahasa pemrograman Java.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Doubly Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada Doubly Linked List memiliki tiga bagian utama, yaitu data, pointer next, dan pointer prev.

Bagian data digunakan untuk menyimpan nilai, pointer next digunakan untuk menunjuk node berikutnya, sedangkan pointer prev digunakan untuk menunjuk node sebelumnya. Node pertama disebut **head**, dan node terakhir memiliki pointer next bernilai **null**, sedangkan node pertama memiliki pointer prev bernilai **null**.

Dibandingkan Single Linked List, Doubly Linked List memiliki kelebihan karena traversal dapat dilakukan dari depan maupun dari belakang. Selain itu, proses insertion dan deletion menjadi lebih fleksibel karena setiap node mengetahui node sebelum dan sesudahnya.

Operasi dasar pada Doubly Linked List meliputi :

- Insertion, yaitu proses menambahkan node baru.
- Deletion, yaitu proses menghapus node.
- Traversal, yaitu proses menelusuri seluruh node.
- Searching, yaitu proses mencari data tertentu.

Keunggulan Doubly Linked List adalah kemudahan dalam navigasi dua arah dan efisiensi dalam manipulasi node.

2.2 Implementasi Program

Berikut adalah beberapa implementasi *Doubly Linked List* :

2.2.1 Class NodeDLL_2511532028

```
package pekan6_2511532028;

public class NodeDLL_2511532028 {
    // mendefinisikan kelas Node
    int data_2028;
    NodeDLL_2511532028 next_2028;
    NodeDLL_2511532028 prev_2028;

    // konstruktor
```

```

public NodeDLL_2511532028(int data) {
    this.data_2028 = data;
    this.next_2028 = null;
    this.prev_2028 = null;
}
}

```

Penjelasan :

Class ini merupakan class dasar yang digunakan untuk membentuk node pada Doubly Linked List.

```
int data_2028;
```

- Variabel ini digunakan untuk menyimpan data pada node.
- Tipe data yang digunakan adalah integer.

```
NodeDLL_2511532028 next_2028;
```

- Variabel ini merupakan pointer.
- Pointer digunakan untuk menunjuk node berikutnya.

```
NodeDLL_2511532028 prev_2028;
```

- Variabel ini merupakan pointer prev.
- Pointer ini digunakan untuk menunjuk node sebelumnya.

```

public NodeDLL_2511532028(int data) {
    this.data_2028 = data;
    this.next_2028 = null;
    this.prev_2028 = null;
}

```

- Method ini merupakan konstruktor.
- Konstruktor akan otomatis dijalankan ketika object node dibuat.
- Data yang dikirim ke konstruktor disimpan ke variabel data_2028.
- Pointer next diisi null.
- Artinya node baru belum terhubung ke node setelahnya.
- ☐ Pointer prev diisi null.
- ☐ Artinya node baru belum terhubung ke node sebelumnya.

2.2.2 Class InsertDLL_2511532028

```

package pekan6_2511532028;

public class InsertDLL_2511532028 {

```

```

static NodeDLL_2511532028 InsertBegin_2028(NodeDLL_2511532028 head, int
data) {
    // buat mode baru
    NodeDLL_2511532028 new_node_2028 = new NodeDLL_2511532028(data);

    //
    new_node_2028.next_2028 = head;
    // jadikan printer prev head ke new node
    if (head != null) {
        head.prev_2028 = new_node_2028;
    }

    return new_node_2028;
}

// fungsi menambahkan node di akhir
public static NodeDLL_2511532028 InsertEnd_2028(NodeDLL_2511532028 head,
int newData) {
    // buat node baru
    NodeDLL_2511532028 newNode_2028 = new NodeDLL_2511532028(newData);
    // jika dll null jadikan head
    if (head == null) {
        head = newNode_2028;
    }
    else {
        NodeDLL_2511532028 curr_2028 = head;
        while (curr_2028.next_2028 != null) {
            curr_2028 = curr_2028.next_2028;
        }
        curr_2028.next_2028 = newNode_2028;
        newNode_2028.prev_2028 = curr_2028;
    }
    return head;
}

// fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511532028 InsertAtPosition_2028(NodeDLL_2511532028
head, int pos, int new_data) {
    // buat node baru
    NodeDLL_2511532028 new_node_2028 = new NodeDLL_2511532028(new_data);

```

```

        if (pos == 1) {
            new_node_2028.next_2028 = head;
            if (head != null) {
                head.prev_2028 = new_node_2028;
            }
            head = new_node_2028;
            return head;
        }
        NodeDLL_2511532028 curr_2028 = head;
        for (int i = 1; i < pos - 1 && curr_2028 != null; i++) {
            curr_2028 = curr_2028.next_2028;
        }
        if (curr_2028 == null) {
            System.out.println("Posisi tidak ada");
            return head;
        }
        new_node_2028.prev_2028 = curr_2028;
        new_node_2028.next_2028 = curr_2028.next_2028;
        curr_2028.next_2028 = new_node_2028;
        if (new_node_2028.next_2028 != null) {
            new_node_2028.next_2028.prev_2028 = new_node_2028;
        }
        return head;
    }

    public static void printlist_2028(NodeDLL_2511532028 head) {
        NodeDLL_2511532028 curr_2028 = head;
        while (curr_2028 != null) {
            System.out.print(curr_2028.data_2028 + " <--> ");
            curr_2028 = curr_2028.next_2028;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // membuat dll 2 <->
        NodeDLL_2511532028 head_2028 = new NodeDLL_2511532028(2);
        head_2028.next_2028 = new NodeDLL_2511532028(3);
        head_2028.next_2028.prev_2028 = head_2028;
        head_2028.next_2028.next_2028 = new NodeDLL_2511532028(5);
    }

```



```

        head_2028.next_2028.next_2028.prev_2028 = head_2028.next_2028;
        // Cetak DLL awal
        System.out.print("DLL Awal : ");
        printlist_2028(head_2028);
        // tambal 1 di awal
        head_2028 = InsertBegin_2028(head_2028, 1);
        System.out.print("simpul 1 di tambah di awal : ");
        printlist_2028(head_2028);
        // tambah 6 di akhir
        System.out.print("simpul 6 di tambah di akhir : ");
        int data_2028 = 6;
        head_2028 = InsertEnd_2028(head_2028, data_2028);
        printlist_2028(head_2028);
        // menambah node 4 di posisi 4
        System.out.print("tambah node 4 di posisi 4 : ");
        int data2_2028 = 4;
        int pos_2028 = 4;
        head_2028 = InsertAtPosition_2028(head_2028, pos_2028, data2_2028);
        printlist_2028(head_2028);
    }
}

```

Penjelasan :

Class ini digunakan untuk menambahkan node pada Doubly Linked List.

```

static NodeDLL_2511532028 InsertBegin_2028(NodeDLL_2511532028 head, int data) {
    // buat mode baru
    NodeDLL_2511532028 new_node_2028 = new NodeDLL_2511532028(data);

    //
    new_node_2028.next_2028 = head;
    // jadikan printer prev head ke new node
    if (head != null) {
        head.prev_2028 = new_node_2028;
    }
}

```

```

        return new_node_2028;
    }

```

- Method ini bertipe NodeDLL_2511532028, artinya method akan mengembalikan node yang nantinya menjadi **head baru**. Parameter head merupakan node pertama pada list. Parameter data merupakan data baru yang akan dimasukkan.
- Data yang dikirim melalui parameter data akan disimpan ke dalam node baru melalui konstruktor.
- Pointer next dari node baru diarahkan ke head lama.
- Program mengecek apakah linked list kosong atau tidak.
- Pointer prev dari head lama diarahkan ke node baru.
- Node baru dikembalikan dan dijadikan **head baru**.

```

public static NodeDLL_2511532028 InsertEnd_2028(NodeDLL_2511532028 head, int
newData) {
    // buat node baru
    NodeDLL_2511532028 newNode_2028 = new NodeDLL_2511532028(newData);
    // jika dll null jadikan head
    if (head == null) {
        head = newNode_2028;
    }
    else {
        NodeDLL_2511532028 curr_2028 = head;
        while (curr_2028.next_2028 != null) {
            curr_2028 = curr_2028.next_2028;
        }
        curr_2028.next_2028 = newNode_2028;
        newNode_2028.prev_2028 = curr_2028;
    }
    return head;
}

```

- Method ini digunakan untuk menambahkan node di bagian akhir linked list.

```

while (curr_2028.next_2028 != null) {
    curr_2028 = curr_2028.next_2028;
}

```

- Traversal sampai node terakhir.

```

curr_2028.next_2028 = newNode_2028;

```

```
newNode_2028.prev_2028 = curr_2028;
```

- Node terakhir menunjuk node baru.
- Node baru menunjuk node sebelumnya.

```
public static NodeDLL_2511532028 InsertAtPosition_2028(NodeDLL_2511532028
head, int pos, int new_data) {
    // buat node baru
    NodeDLL_2511532028 new_node_2028 = new NodeDLL_2511532028(new_data);
    if (pos == 1) {
        new_node_2028.next_2028 = head;
        if (head != null) {
            head.prev_2028 = new_node_2028;
        }
        head = new_node_2028;
        return head;
    }
    NodeDLL_2511532028 curr_2028 = head;
    for (int i = 1; i < pos - 1 && curr_2028 != null; i++) {
        curr_2028 = curr_2028.next_2028;
    }
    if (curr_2028 == null) {
        System.out.println("Posisi tidak ada");
        return head;
    }
    new_node_2028.prev_2028 = curr_2028;
    new_node_2028.next_2028 = curr_2028.next_2028;
    curr_2028.next_2028 = new_node_2028;
    if (new_node_2028.next_2028 != null) {
        new_node_2028.next_2028.prev_2028 = new_node_2028;
    }
    return head;
}
```

- Method ini menambahkan data ke posisi tertentu

```
for (int i = 1; i < pos - 1 && curr_2028 != null; i++) {
    curr_2028 = curr_2028.next_2028;
}
```

- Traversal sampai posisi sebelum node yang ingin disisipkan.

```
new_node_2028.prev_2028 = curr_2028;
new_node_2028.next_2028 = curr_2028.next_2028;
```

- Pointer prev node baru.
- Pointer next node baru.

```
curr_2028.next_2028 = new_node_2028;
if (new_node_2028.next_2028 != null) {
    new_node_2028.next_2028.prev_2028 = new_node_2028;
}
```

- Sambungkan node sebelumnya.
- Sambungkan node sesudahnya.

Gambar 2.2.2

```
<terminated> InsertDLL_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse
DLL Awal : 2 <--> 3 <--> 5 <-->
simpul 1 di tambah di awal : 1 <--> 2 <--> 3 <--> 5 <-->
simpul 6 di tambah di akhir : 1 <--> 2 <--> 3 <--> 5 <--> 6 <-->
tambah node 4 di posisi 4 : 1 <--> 2 <--> 3 <--> 4 <--> 5 <--> 6 <-->
```

2.2.3 Class HapusDLL_2511532028

```
package pekan6_2511532028;

public class HapusDLL_2511532028 {
    // fungsi menghapus node awal
    public static NodeDLL_2511532028 delHead_2028(NodeDLL_2511532028 head) {
        if (head == null)
            return null;
        //NodeDLL_2511532028 temp_2028 = head;
        head = head.next_2028;
        if (head != null) {
            head.prev_2028 = null;
        }
        return head;
    }

    // fungsi menghapus di akhir
    public static NodeDLL_2511532028 delLast_2028(NodeDLL_2511532028 head) {
        if (head == null) {
            return null;
        }
        if (head.next_2028 == null) {
            return null;
        }
    }
}
```

```

    }
    NodeDLL_2511532028 curr_2028 = head;
    while (curr_2028.next_2028 != null) {
        curr_2028 = curr_2028.next_2028;
    }
    // update pointer previous node
    if (curr_2028.prev_2028 != null) {
        curr_2028.prev_2028.next_2028 = null;
    }
    return head;
}

public static NodeDLL_2511532028 delPos_2028(NodeDLL_2511532028 head, int
pos) {
    if (head == null) {
        return head;
    }
    NodeDLL_2511532028 curr_2028 = head;
    for (int i = 1; curr_2028 != null && i < pos; ++i) {
        curr_2028 = curr_2028.next_2028;
    }
    if (curr_2028 == null) {
        return head;
    }

    if (curr_2028.prev_2028 != null) {
        curr_2028.prev_2028.next_2028 = curr_2028.next_2028;
    }
    if (curr_2028.next_2028 != null) {
        curr_2028.next_2028.prev_2028 = curr_2028.prev_2028;
    }

    if (head == curr_2028) {
        head = curr_2028.next_2028;
    }
    return head;
}

public static void printlist_2028(NodeDLL_2511532028 head) {
    NodeDLL_2511532028 curr_2028 = head;

```

```

        while (curr_2028 != null) {
            System.out.print(curr_2028.data_2028 + " <--> ");
            curr_2028 = curr_2028.next_2028;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeDLL_2511532028 head_2028 = new NodeDLL_2511532028(1);
        head_2028.next_2028 = new NodeDLL_2511532028(2);
        head_2028.next_2028.prev_2028 = head_2028;
        head_2028.next_2028.next_2028 = new NodeDLL_2511532028(3);
        head_2028.next_2028.next_2028.prev_2028 = head_2028.next_2028;
        head_2028.next_2028.next_2028.next_2028 = new NodeDLL_2511532028(4);
        head_2028.next_2028.next_2028.next_2028.prev_2028 =
head_2028.next_2028.next_2028;
        head_2028.next_2028.next_2028.next_2028.next_2028 = new
NodeDLL_2511532028(5);
        head_2028.next_2028.next_2028.next_2028.next_2028.prev_2028 =
head_2028.next_2028.next_2028.next_2028;

        System.out.print("DLL Awal : ");
        printlist_2028(head_2028);

        System.out.print("Setelah head di hapus : ");
        head_2028 = delHead_2028(head_2028);
        printlist_2028(head_2028);

        System.out.print("Setelah node terakhir di hapus : ");
        head_2028 = dellast_2028(head_2028);
        printlist_2028(head_2028);

        System.out.print("menghapus node ke-2 : ");
        head_2028 = delPos_2028(head_2028, 2);

        printlist_2028(head_2028);
    }
}

```

Penjelasan :

- Class untuk menghapus node.

```
public static NodeDLL_2511532028 delHead_2028(NodeDLL_2511532028 head) {  
    if (head == null)  
        return null;  
    //NodeDLL_2511532028 temp_2028 = head;  
    head = head.next_2028;  
    if (head != null) {  
        head.prev_2028 = null;  
    }  
    return head;  
}
```

- Method untuk menghapus head / nilai depan.
- Head dipindahkan ke node berikutnya.
- Pointer prev head baru diputus.

```
public static NodeDLL_2511532028 delLast_2028(NodeDLL_2511532028 head) {  
    if (head == null) {  
        return null;  
    }  
    if (head.next_2028 == null) {  
        return null;  
    }  
    NodeDLL_2511532028 curr_2028 = head;  
    while (curr_2028.next_2028 != null) {  
        curr_2028 = curr_2028.next_2028;  
    }  
    // update pointer previous node  
    if (curr_2028.prev_2028 != null) {  
        curr_2028.prev_2028.next_2028 = null;  
    }  
    return head;  
}
```

- Method untuk menghapus nilai terakhir.

```
while (curr_2028.next_2028 != null) {  
    curr_2028 = curr_2028.next_2028;  
}  
// update pointer previous node  
if (curr_2028.prev_2028 != null) {
```

```
curr_2028.prev_2028.next_2028 = null;
}
```

- Traversal sampai node terakhir.
- Pointer next node sebelumnya diputus.

```
public static NodeDLL_2511532028 delPos_2028(NodeDLL_2511532028 head, int
pos) {
    if (head == null) {
        return head;
    }
    NodeDLL_2511532028 curr_2028 = head;
    for (int i = 1; curr_2028 != null && i < pos; ++i) {
        curr_2028 = curr_2028.next_2028;
    }
    if (curr_2028 == null) {
        return head;
    }

    if (curr_2028.prev_2028 != null) {
        curr_2028.prev_2028.next_2028 = curr_2028.next_2028;
    }
    if (curr_2028.next_2028 != null) {
        curr_2028.next_2028.prev_2028 = curr_2028.prev_2028;
    }

    if (head == curr_2028) {
        head = curr_2028.next_2028;
    }
    return head;
}
```

- Menghapus node / data pada posisi tertentu.

```
if (curr_2028.prev_2028 != null) {
    curr_2028.prev_2028.next_2028 = curr_2028.next_2028;
}
if (curr_2028.next_2028 != null) {
    curr_2028.next_2028.prev_2028 = curr_2028.prev_2028;
}
```

- Node sebelumnya melompati node target.
- Node setelahnya menunjuk node sebelumnya.


```

public static void printlist_2028(NodeDLL_2511532028 head) {
    NodeDLL_2511532028 curr_2028 = head;
    while (curr_2028 != null) {
        System.out.print(curr_2028.data_2028 + " <--> ");
        curr_2028 = curr_2028.next_2028;
    }
    System.out.println();
}

```

- Method untuk menampilkan node / data pada list menggunakan traversal.

Output :

Gambar 2.2.3

```

<terminated> HapusDLL_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.e
DLL Awal : 1 <--> 2 <--> 3 <--> 4 <--> 5 <-->
Setelah head di hapus : 2 <--> 3 <--> 4 <--> 5 <-->
Setelah node terakhir di hapus : 2 <--> 3 <--> 4 <-->
menghapus node ke-2 : 2 <--> 4 <-->

```

2.2.4 Class PenelusuranDLL_2511532028

```

package pekan6_2511532028;

public class PenelusuranDLL_2511532028 {
    static void forwardTraversal_2028(NodeDLL_2511532028 head) {
        // memulai penelusuran dari head
        NodeDLL_2511532028 curr_2028 = head;
        // lanjutkan sampai akhir
        while (curr_2028 != null) {
            // print data
            System.out.print(curr_2028.data_2028 + " <--> ");
            // pindah ke node berikutnya
            curr_2028 = curr_2028.next_2028;
        }
        System.out.println();
    }

    // fungsi penelusuran mundur
    static void backwardTraversal_2028(NodeDLL_2511532028 tail) {
        // mulai dari akhir
    }
}

```

```

        NodeDLL_2511532028 curr_2028 = tail;
        // lanjut
        while (curr_2028 != null) {
            // cetak data
            System.out.print(curr_2028.data_2028 + " <--> ");
            // pindah ke node sebelumnya
            curr_2028 = curr_2028.prev_2028;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // cetak DLL
        NodeDLL_2511532028 head_2028 = new NodeDLL_2511532028(1);
        NodeDLL_2511532028 second_2028 = new NodeDLL_2511532028(2);
        NodeDLL_2511532028 third_2028 = new NodeDLL_2511532028(3);

        head_2028.next_2028 = second_2028;
        second_2028.prev_2028 = head_2028;
        second_2028.next_2028 = third_2028;
        third_2028.prev_2028 = second_2028;

        System.out.println("Penelusuran maju : ");
        forwardTraversal_2028(head_2028);

        System.out.println("Penelusuran mundur : ");
        backwardTraversal_2028(third_2028);
    }
}

```

Penjelasan :

```

public class PenelusuranDLL_2511532028 {
    static void forwardTraversal_2028(NodeDLL_2511532028 head) {
        // memulai penelusuran dari head
        NodeDLL_2511532028 curr_2028 = head;
        // lanjutkan sampai akhir
        while (curr_2028 != null) {
            // print data
            System.out.print(curr_2028.data_2028 + " <--> ");

```

```

        // pindah ke node berikutnya
        curr_2028 = curr_2028.next_2028;
    }
    System.out.println();
}

```

- Method ini menampilkan node pada list dari node terdepan hingga akhir (forward).

```

while (curr_2028 != null) {
    //print data
    System.out.print(curr_2028.data_2028 + " <--> ");
    // pindah ke node berikutnya
    curr_2028 = curr_2028.next_2028;
}

```

- Forward traversal

```

static void backwardTraversal_2028(NodeDLL_2511532028 tail) {
    // mulai dari akhir
    NodeDLL_2511532028 curr_2028 = tail;
    // lanjut
    while (curr_2028 != null) {
        // cetak data
        System.out.print(curr_2028.data_2028 + " <--> ");
        // pindah ke node sebelumnya
        curr_2028 = curr_2028.prev_2028;
    }
    System.out.println();
}

```

- Menampilkan node dari paling belakang hingga node terdepan (backward/reverse).

```

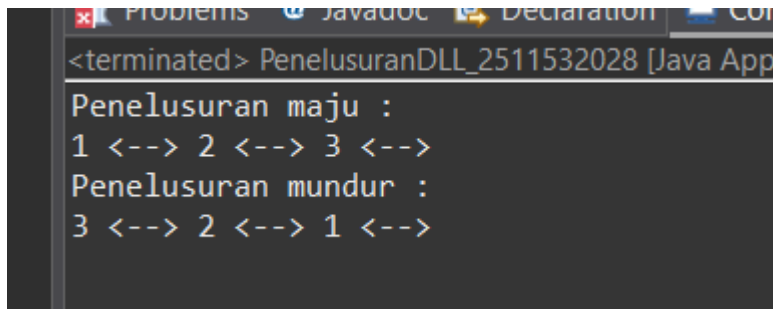
while (curr_2028 != null) {
    // cetak data
    System.out.print(curr_2028.data_2028 + " <--> ");
    // pindah ke node sebelumnya
    curr_2028 = curr_2028.prev_2028;
}

```

- Backward traversal

Output :

Gambar 2.2.4



2.3 Tugas Praktikum

2.3.1 Class Lagu_2511532028

```
package pekan6_2511532028;

public class Lagu_2511532028 {
    String judul_2028;
    String penyanyi_2028;
    Lagu_2511532028 next_2028;
    Lagu_2511532028 prev_2028;

    public Lagu_2511532028(String judul, String penyanyi) {
        this.judul_2028 = judul;
        this.penyanyi_2028 = penyanyi;
        this.next_2028 = null;
        this.prev_2028 = null;
    }

    public void SetJudul_2028(String judul) {this.judul_2028 = judul;}
    public void SetPenyanyi_2028(String penyanyi) {this.penyanyi_2028 =
penyanyi;}
    public String GetJudul_2028() {return judul_2028;}
    public String GetPenyanyi_2028() {return penyanyi_2028;}
}
```

Penjelasan :

Class ini digunakan sebagai node pada Doubly Linked List. Setiap node menyimpan data judul lagu dan penyanyi.

Setiap node memiliki beberapa bagian :

- Judul lagu

- Penyanyi dari lagu tersebut
- Pointer ke node berikutnya
- Pointer ke node sebelumnya

2.3.2 Class Musik_2511532028

```
package pekan6_2511532028;

import java.util.Scanner;

public class Musik_2511532028 {
    Scanner i = new Scanner(System.in);

    public static Lagu_2511532028 tambahLagu_2028(Lagu_2511532028 lagu, Scanner
i) {
        System.out.println("=====Tambah Lagu=====");
        String judul_2028;
        String penyanyi_2028;

        System.out.print("Judul Lagu      : ");
        judul_2028 = i.nextLine();
        System.out.print("Nama Penyanyi  : ");
        penyanyi_2028 = i.nextLine();
        Lagu_2511532028 newLagu_2028 = new Lagu_2511532028(judul_2028,
penyanyi_2028);
        if (lagu == null) {
            return newLagu_2028;
        }
        Lagu_2511532028 tail_2028 = lagu;

        while (tail_2028.next_2028 != null) {
            tail_2028 = tail_2028.next_2028;
        }

        tail_2028.next_2028 = newLagu_2028;
        newLagu_2028.prev_2028 = tail_2028;
        System.out.println("====Lagu Berhasil Ditambahkan====");
        return lagu;
    }
}
```

```

public static Lagu_2511532028 hapusLaguAwal_2028(Lagu_2511532028 lagu) {
    if (lagu == null ) {
        System.out.println("List lagu kosong");
        return null;
    }
    Lagu_2511532028 hapuslagu_2028 = lagu;
    System.out.println("====Hapus Lagu====");
    System.out.println("Lagu   : " + hapuslagu_2028.judul_2028
        + "\nPenyanyi: " + hapuslagu_2028.penyanyi_2028);
    System.out.println("====Berhasil dihapus====");
    hapuslagu_2028 = hapuslagu_2028.next_2028;
    if (hapuslagu_2028 != null) {
        hapuslagu_2028.prev_2028 = null;
    }
    return hapuslagu_2028;
}

public static void tampilMaju_2028(Lagu_2511532028 lagu) {
    if (lagu == null) {
        System.out.println("List Playlist kosong");
        return;
    }
    System.out.println("====List Playlist Lagu Dari
Awal====");
    Lagu_2511532028 currlagu_2028 = lagu;
    while (currlagu_2028 != null) {
        System.out.print(currlagu_2028.judul_2028 + " " +
currlagu_2028.penyanyi_2028 + " <--> ");
        currlagu_2028 = currlagu_2028.next_2028;
    }
    System.out.println();
}

public static void tampilMundur_2028(Lagu_2511532028 lagu) {
    if (lagu == null) {
        System.out.println("List Playlist kosong");
        return;
    }
    System.out.println("====List Playlist Lagu Dari
Akhir====");

```

```

        Lagu_2511532028 back_2028 = lagu;

        while (back_2028.next_2028 != null) {
            back_2028 = back_2028.next_2028;
        }

        while ( back_2028 != null) {
            System.out.print(back_2028.judul_2028 + " " +
back_2028.penyanyi_2028 + " <--> ");
            back_2028 = back_2028.prev_2028;
        }
        System.out.println();
    }

    public static boolean cariLagu_2028(Lagu_2511532028 lagu, Scanner i) {
        if (lagu == null) {return false;}
        Lagu_2511532028 currlagu_2028 = lagu;
        String judul_2028;
        System.out.println("====Cari Lagu====");
        System.out.print("Judul Lagu      : ");
        judul_2028 = i.nextLine();
        while (currlagu_2028 != null) {
            if (judul_2028.equalsIgnoreCase(currlagu_2028.judul_2028)) {
                System.out.println("Judul Lagu      : " +
currlagu_2028.judul_2028);
                System.out.println("Penyanyi Lagu      : " +
currlagu_2028.penyanyi_2028);
                return true;
            }
            currlagu_2028 = currlagu_2028.next_2028;
        }
        return false;
    }

    public static void MenuMusik_2028() {
        System.out.println("\n====Menu Playlist Musik====");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Liat Playlist (dari awal)");
        System.out.println("4. Liat Playlist (dari akhir)");
    }

```

```

        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");

    }

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        Lagu_2511532028 Lagu_2028 = null;
        int pilihan_2028;
        do {
            MenuMusik_2028();
            System.out.print("Pilih menu : ");
            pilihan_2028 = s.nextInt();
            System.out.println();
            s.nextLine();

            switch (pilihan_2028) {
                case 1:
                    Lagu_2028 = tambahLagu_2028(Lagu_2028,s);
                    break;
                case 2:
                    Lagu_2028 = hapusLaguAwal_2028(Lagu_2028);
                    break;
                case 3:
                    tampilMaju_2028(Lagu_2028);
                    break;
                case 4:
                    tampilMundur_2028(Lagu_2028);
                    break;
                case 5:
                    if (cariLagu_2028(Lagu_2028,s)) {
                        System.out.println("====ketemu====");
                    } else {
                        System.out.println("====tidak ketemu atau
list kosong====");
                    }
                    break;
                case 6:
                    System.out.println("===Anda Keluar dari Menu
Playlist===");
            }
        }
    }
}

```



```

        break;
    }

    } while (pilihan_2028 != 6);

}

}

```

Penjelasan :

```

public static Lagu_2511532028 tambahLagu_2028(Lagu_2511532028 lagu, Scanner
i) {

    System.out.println("====Tambah Lagu====");
    String judul_2028;
    String penyanyi_2028;

    System.out.print("Judul Lagu      : ");
    judul_2028 = i.nextLine();
    System.out.print("Nama Penyanyi  : ");
    penyanyi_2028 = i.nextLine();
    Lagu_2511532028 newLagu_2028 = new Lagu_2511532028(judul_2028,
penyanyi_2028);
    if (lagu == null) {
        return newLagu_2028;
    }
    Lagu_2511532028 tail_2028 = lagu;

    while (tail_2028.next_2028 != null) {
        tail_2028 = tail_2028.next_2028;
    }

    tail_2028.next_2028 = newLagu_2028;
    newLagu_2028.prev_2028 = tail_2028;
    System.out.println("====Lagu Berhasil Ditambahkan====");
    return lagu;

}

```

- Method ini digunakan untuk menambahkan lagu ke akhir playlist.

```

Lagu_2511532028 newLagu_2028 = new Lagu_2511532028(judul_2028,
penyanyi_2028);

```

- Membuat node baru.

```
if (lagu == null) {
    return newLagu_2028;
}
```

- Jika playlist kosong, node baru langsung menjadi head

```
while (tail_2028.next_2028 != null) {
    tail_2028 = tail_2028.next_2028;
}
```

- Bergerak sampai node terakhir.

```
tail_2028.next_2028 = newLagu_2028;
newLagu_2028.prev_2028 = tail_2028;
```

- Node terakhir menunjuk ke node baru.
- Node baru menunjuk kembali ke node sebelumnya.

```
public static Lagu_2511532028 hapusLaguAwal_2028(Lagu_2511532028 lagu) {
    if (lagu == null ) {
        System.out.println("List lagu kosong");
        return null;
    }
    Lagu_2511532028 hapuslagu_2028 = lagu;
    System.out.println("=====Hapus Lagu=====");
    System.out.println("Lagu   : " + hapuslagu_2028.judul_2028
        + "\nPenyanyi: " + hapuslagu_2028.penyanyi_2028);
    System.out.println("=====Berhasil dihapus=====");
    hapuslagu_2028 = hapuslagu_2028.next_2028;
    if (hapuslagu_2028 != null) {
        hapuslagu_2028.prev_2028 = null;
    }
    return hapuslagu_2028;
}
```

- Method ini untuk menghapus lagu di awal list.

```
Lagu_2511532028 hapuslagu_2028 = lagu;
```

- Pointer bantu menunjuk head.

```
hapuslagu_2028 = hapuslagu_2028.next_2028;
```

- Head digeser ke node berikutnya.

```
if (hapuslagu_2028 != null) {
    hapuslagu_2028.prev_2028 = null;
}
```

- Prev dari head baru dibuat null.

```

public static void tampilMaju_2028(Lagu_2511532028 lagu) {
    if (lagu == null) {
        System.out.println("List Playlist kosong");
        return;
    }
    System.out.println("=====List Playlist Lagu Dari
Awal=====");
    Lagu_2511532028 currlagu_2028 = lagu;
    while (currlagu_2028 != null) {
        System.out.print(currlagu_2028.judul_2028 + " " +
currlagu_2028.penyanyi_2028 + " <--> ");
        currlagu_2028 = currlagu_2028.next_2028;
    }
    System.out.println();
}

```

- Method menampilkan list dari terdepan.
- Mulai dari head.
- Bergerak dengan next.

```

public static void tampilMundur_2028(Lagu_2511532028 lagu) {
    if (lagu == null) {
        System.out.println("List Playlist kosong");
        return;
    }
    System.out.println("=====List Playlist Lagu Dari
Akhir=====");
    Lagu_2511532028 back_2028 = lagu;

    while (back_2028.next_2028 != null) {
        back_2028 = back_2028.next_2028;
    }

    while ( back_2028 != null) {
        System.out.print(back_2028.judul_2028 + " " +
back_2028.penyanyi_2028 + " <--> ");
        back_2028 = back_2028.prev_2028;
    }
    System.out.println();
}

```

- Method menampilkan list dari node paling belakang.

```
while (back_2028.next_2028 != null) {
    back_2028 = back_2028.next_2028;
}
```

- Mencari node paling belakang.

```
while ( back_2028 != null) {
    System.out.print(back_2028.judul_2028 + " " +
back_2028.penyanyi_2028 + " <--> ");
    back_2028 = back_2028.prev_2028;
}
```

- Bergerak mundur memakai prev.

Gambar 2.3.2

Menambahkan Lagu dan Penyanyinya :

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins

====Menu Playlist Musik====
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 1

=====Tambah Lagu=====
Judul Lagu      : Bunga Abadi
Nama Penyanyi   : Rio Clappy
```

Menampilkan playlist lagu dari terdepan :

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v2025
nama : Penyanyi : Aku Jeje
====Lagu Berhasil Ditambahkan====

====Menu Playlist Musik====
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 3

=====List Playlist Lagu Dari Awal=====
Bunga Abadi-Rio Clappy <--> Gemuruh Riuh-Mighfar Suganda <--> Taman Bunga-Aku Jeje <-->
```


Menampilkan playlist lagu dari paling belakang :

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
=====List Playlist Lagu Dari Akhir=====
Bunga Abadi-Rio Clappy <--> Gemuruh Riu-Mighfar Suganda <--> Taman Bunga-Aku Jeje <-->

====Menu Playlist Musik=====
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 4

=====List Playlist Lagu Dari Akhir=====
Taman Bunga-Aku Jeje <--> Gemuruh Riu-Mighfar Suganda <--> Bunga Abadi-Rio Clappy <-->
```

Menghapus lagu paling terdepan:

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
6. Keluar
Pilih menu : 2

=====Hapus Lagu=====
Lagu      : Bunga Abadi
Penyanyi: Rio Clappy
=====Berhasil dihapus=====
```

Mencari judul lagu :

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 5

=====Cari Lagu=====
Judul Lagu      : taman bunga
Judul Lagu      : Taman Bunga
Penyanyi Lagu   : Aku Jeje
====ketemu=====
```

Saat playlist masih kosong :

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugin
=====Menu Playlist Musik=====
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 2

List lagu kosong
```

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugin
=====Menu Playlist Musik=====
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 3

List Playlist kosong
```

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool\plugin
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 4

List Playlist kosong

=====Menu Playlist Musik=====
```

```
Musik_2511532028 [Java Application] C:\Users\USER\p2\pool
2. Hapus Lagu (dari awal)
3. Liat Playlist (dari awal)
4. Liat Playlist (dari akhir)
5. Cari Lagu
6. Keluar
Pilih menu : 5

====tidak ketemu atau list kosong====
```

Program playlist musik ini berhasil mengimplementasikan **Doubly Linked List**, di mana setiap lagu memiliki pointer next dan prev. Program dapat melakukan operasi :

- menambah lagu,
- menghapus lagu,
- traversal maju,
- traversal mundur,
- searching lagu berdasarkan judul (tidak case-sensitive).

Dengan penggunaan pointer prev, proses penelusuran dari belakang dapat dilakukan dengan lebih efisien dibandingkan Single Linked List.

BAB III

PENUTUP

3.1 Kesimpulan

Doubly Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung menggunakan dua buah pointer, yaitu pointer yang menunjuk ke node berikutnya dan pointer yang menunjuk ke node sebelumnya. Dengan adanya dua pointer tersebut, proses traversal dapat dilakukan secara dua arah, baik dari depan ke belakang maupun dari belakang ke depan.

Melalui praktikum ini, dapat dipahami bagaimana proses pembentukan node, penambahan data, pencarian data, penghapusan data, serta proses traversal pada Doubly Linked List menggunakan bahasa pemrograman Java.

Dengan memahami konsep Doubly Linked List, mahasiswa diharapkan mampu mengimplementasikan struktur data dinamis secara lebih efektif untuk menyelesaikan berbagai permasalahan dalam pemrograman.

3.2 Saran

Dalam mempelajari Doubly Linked List, disarankan untuk memahami terlebih dahulu konsep dasar node, pointer, head, dan tail sebelum mengimplementasikan program.

Selain itu, latihan yang beragam seperti insertion, deletion, searching, serta traversal maju dan mundur akan membantu meningkatkan pemahaman terhadap cara kerja Doubly Linked List.

Mahasiswa juga disarankan untuk mencoba mengimplementasikan Doubly Linked List dalam berbagai studi kasus nyata agar pemahaman terhadap struktur data ini menjadi lebih mendalam.

Referensi

Data Structures and Algorithms in Java

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th Edition). Wiley.

Oracle. *Java Collections Framework*.